Activation Functions in Deep Learning

Sigmoid · Tanh · ReLU — why they exist, how they work, and when to use each

1. Why Activation Functions?

A neural network is built by chaining layers of neurons, each computing a **weighted sum** of its inputs. Without an activation function, every layer is just a linear transformation — and any sequence of linear transformations collapses into a *single* linear transformation. No matter how many layers you stack, the network can only model straight-line relationships.

Activation functions introduce **non-linearity** after every neuron's computation. This is what allows deep networks to learn curves, edges, speech patterns, and every complex real-world signal that cannot be described by a line.

Core Intuition

A neuron computes: **output = activation(w■x■ + w■x■ + ... + b)**. The activation function is the gatekeeper — it decides how much of that sum to pass forward, and in what shape.

2. Sigmoid Function

Formula

$$\sigma(x) = 1 / (1 + e^{-x})$$

mooth & differentiable | ✓ Output looks like probability | ✗ Vanishing gradient | ✗ Not zero-centered | ✗ Computationally heavy

How it works

Sigmoid maps any real number to the range **(0, 1)**. Think of it as a "confidence meter": values near 1 mean strongly positive, near 0 mean strongly negative, and 0.5 means uncertain. The curve is S-shaped (hence the name) and smooth everywhere.

The vanishing gradient problem: For very large or very small inputs, the Sigmoid curve becomes nearly flat. The gradient (slope) at those points approaches zero. During backpropagation, gradients are multiplied layer by layer — if each is near zero, the signal reaching early layers becomes infinitesimally small. Early layers stop learning. This makes Sigmoid unsuitable for hidden layers in deep networks.

When to use Sigmoid

Output layer of **binary classification** networks (e.g. spam/not-spam, disease/no-disease). The output naturally represents probability. **Avoid** in hidden layers of any deep network.

3. Tanh (Hyperbolic Tangent)

Formula

$$\tanh(x) = (e^{x} - e^{-x}) / (e^{x} + e^{-x})$$

✓ Zero-centered output | ✓ Stronger gradient than Sigmoid | ✗ Still saturates | ✗ Vanishing gradient in deep nets

How it works

Tanh is essentially a **rescaled Sigmoid**. It maps inputs to the range **(–1, +1)** and its shape is also S-curved. The key difference is that it is **zero-centered**: outputs can be negative, zero, or positive.

Why zero-centering matters:

Sigmoid (not zero-centered) All outputs are between 0 and 1, so all gradients flowing back from a layer point in the same direction (all positive or all negative). This forces *zig-zag* updates during gradient descent, slowing convergence.	**Tanh (zero-centered)** Outputs range from –1 to +1, so gradients can be positive or negative. This allows the optimizer to move more efficiently in any direction, leading to faster and smoother learning.

Tanh also has a **stronger maximum gradient (~1.0)** compared to Sigmoid's maximum of only 0.25, so error signals flow back more easily. However, it still saturates for extreme inputs, so the vanishing gradient problem is reduced but not eliminated.

> **When to use Tanh**
> Hidden layers of **RNNs and LSTMs**, where zero-centered outputs matter for sequence learning. Better than Sigmoid for hidden layers, but ReLU is usually preferred in feedforward networks.

4. ReLU — Rectified Linear Unit

Formula

```
ReLU(x) = max(0, x)
```

o saturation for positives | ✓ Computationally trivial | ✓ Sparse activation | ✗ Dying ReLU problem | ✗ Not zero-centered

How it works

ReLU is elegantly simple: if the input is negative, output is 0. If positive, output equals the input. Despite this simplicity, it **revolutionised deep learning** and remains the default activation function in most modern architectures.

Why ReLU dominates:

No vanishing gradient (positive side)	For any positive input, the gradient is exactly 1 — no shrinkage. Gradients flow back through any number of layers without dying.
Speed	Computing max(0, x) is a single comparison — ~6× faster than Sigmoid or Tanh which require exponential calculations.
Sparsity	Negative inputs output 0, so typically only ~50% of neurons are active at once. Sparse networks generalise better and use less memory.

The Dying ReLU problem: If a neuron's weights get updated such that its input is always negative (e.g. after a large bad gradient update), it permanently outputs 0 and its gradient is always 0. It "dies" and never recovers.

Solutions include **Leaky ReLU** (a small slope for negatives, e.g. 0.01x), **Parametric ReLU**, and **ELU** — all covered in Day 28.

> **When to use ReLU**
> **Default choice** for hidden layers in CNNs, feedforward networks, and most modern architectures. If neurons are dying, switch to Leaky ReLU. Never use in output layers for classification.

5. Visual Comparison

Activation curves — output vs input for all three functions

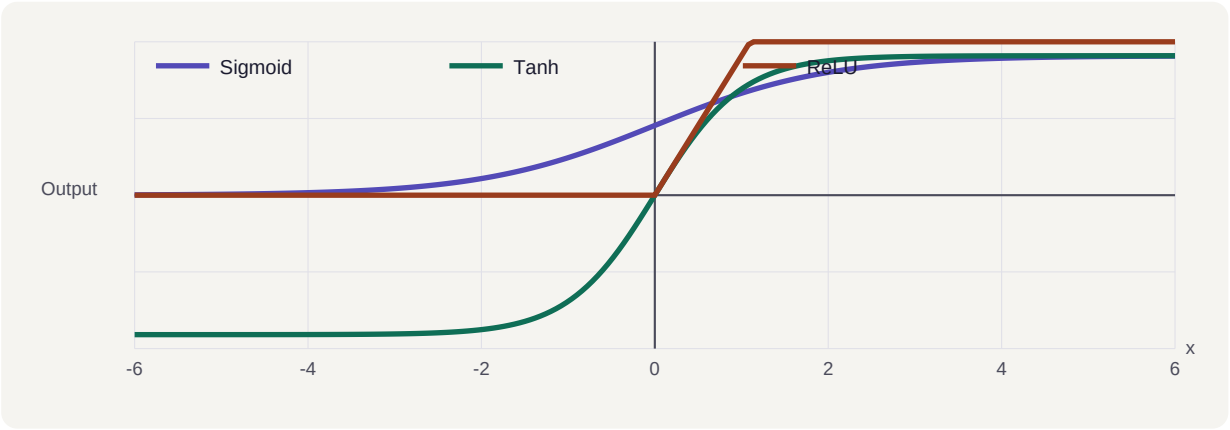

Figure 1: Sigmoid (purple) saturates between 0–1. Tanh (teal) saturates between –1 and +1. ReLU (coral) is linear for positive inputs and zero elsewhere.

Gradient curves — the learning signal at each input value

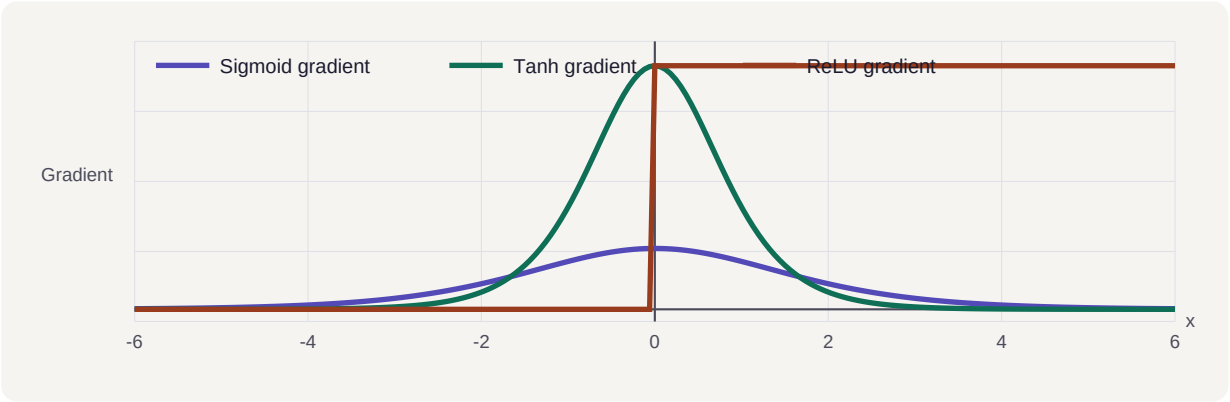

Figure 2: Sigmoid gradient peaks at only ~0.25. Tanh peaks at 1.0. ReLU is a constant 1 for all positive inputs — explaining why it avoids the vanishing gradient problem.

6. Side-by-Side Summary

Property	Sigmoid	Tanh	ReLU
Output range	(0, 1)	(–1, +1)	[0, ∞)
Zero-centered?	✗ No	✓ Yes	✗ No

Property	Sigmoid	Tanh	ReLU
Max gradient	0.25	1.0	1.0 (pos.)
Vanishing gradient?	✗ Severe	~ Moderate	✓ No (pos.)
Computation speed	Slow	Slow	✓ Very fast
Dying neurons?	No	No	✗ Yes (neg.)
Best used in	Output (binary)	RNNs / LSTMs	Hidden layers

7. Quick Decision Guide

Binary classification output	Use Sigmoid. The (0,1) range maps naturally to probability.
Recurrent network (RNN / LSTM)	Use Tanh. Zero-centering helps sequence learning; Tanh is built into LSTM gates.
Hidden layers — any feedforward or CNN	Use ReLU by default. Fast, effective, no vanishing gradient on the positive side.
Neurons dying / ReLU not working	Switch to Leaky ReLU, Parametric ReLU, or ELU (covered in Day 28).

Key Takeaways

1. Activation functions are essential for non-linearity — without them, deep networks collapse to one linear layer.
2. Sigmoid was the pioneer: smooth and probabilistic, but kills gradients in deep networks.
3. Tanh improved on Sigmoid with zero-centered outputs and stronger gradients, but still saturates.
4. ReLU revolutionised the field with its simplicity: constant gradient for positives, lightning-fast computation.
5. The vanishing gradient problem is why ReLU (and its variants) dominate modern architecture design.
6. ReLU variants (Leaky ReLU, PReLU, ELU) solve the Dying ReLU problem and are the next step to explore.